



SQL en BigQuery

Mundo de Datos Mex

por Erick Orlando Matla Cruz



Estructura de una consulta

SELECT <lista_atributos>

FROM <nombre_proyecto.nombre_conjunto.nombre_tabla>;

donde:

SELECT permite indicar el listado de atributos que desea obtenerse del conjunto de datos especificado en FROM , puede utilizarse “*” para indicar que se desea recuperar todos los atributos

FROM permite indicar la tabla o relación que desea consultar



Estructura de una consulta

nombre_proyecto puede ser omitido, obligatorio cuando se trabaja con conjuntos de datos de diferentes proyectos

nombre_conjunto deberá estar presente y es el nombre del conjunto de datos al que pertenece la tabla que desea manipularse

nombre_tabla es la tabla que contiene los datos que se desea manipular y/o recuperar



Condiciones sobre tuplas

SELECT <lista_atributos>

FROM <nombre_proyecto.nombre_conjunto.nombre_tabla>

WHERE <atributo> <condición> <valor/valores>;

donde:

WHERE permite especificar una condición que deben cumplir las tuplas especificadas en FROM



Condiciones sobre tuplas

<atributo> permite especificar un atributo de la relación especificada en FROM, éste puede ser invocado por una función de cadena, fecha, numérica y/o CAST

<condición> permite indicar el tipo de comparación a realizar, puede ser =, !=, >=, <=, LIKE, NOT LIKE, IN, NOT IN

<valor/valores> es el valor contra el que se realizará la comparación en la condición, puede ser un conjunto de datos formado por el resultado de una consulta que arroja un atributo en la cláusula SELECT (subconsulta)



Subconsultas

SELECT <lista_atributos>

FROM (SELECT <lista_atributos>

FROM <nombre_proyecto.nombre_conjunto.nombre_tabla>) T1;

donde

La tabla a consultar es resultado de una consulta, para poder utilizar el resultado de una consulta como la relación especificada en FROM, se debe renombrar



Subconsultas

```
SELECT <lista_atributos>  
FROM <nombre_proyecto.nombre_conjunto.nombre_tabla>  
WHERE <atributo> = (SELECT <atributo>  
                     FROM  
                     <nombre_proyecto.nombre_conjunto.nombre_tabla>);
```

donde:

el resultado de la consulta es un **ÚNICO VALOR**



Funciones de agregación

Permiten obtener un único valor como resultado de aplicar la función de agregación a un conjunto de datos. En SQL se considera conjunto de datos, a los valores que se encuentran en un único atributo de una tabla.

La estructura básica para utilizar una función de agregación es:

```
SELECT función_agregación<atributo>
```

```
FROM <nombre_proyecto.nombre_conjunto.nombre_tabla>;
```




Funciones de agregación

donde:

función_agregación puede ser MAX,MIN,SUM,COUNT,AVG

<atributo> es el nombre de UN SOLO ATRIBUTO de la tabla a consultar

con este tipo de consulta, obtendremos un único valor como resultado, por ejemplo, el valor máximo de un conjunto de datos.



Funciones de agregación y agrupaciones

Las agrupaciones permiten crear “subgrupos” de tuplas dados los valores de un atributo, esto es, se crearán tantos subgrupos de tuplas como valores diferentes existan en el atributo especificado en la agrupación.

La estructura básica para utilizar una agrupación es:

```
SELECT <lista_atributos>
```

```
FROM <nombre_proyecto.nombre_conjunto.nombre_tabla>
```

```
GROUP BY <lista_atributos>;
```



Funciones de agregación

donde:

<lista_atributos> indica los atributos que permitirán generar los grupos de tuplas, por cada combinación de valores diferentes entre estos atributos, se genera un grupo. Todos los atributos que aparecen aquí, deben aparecer en GROUP BY o bien, deben ser utilizados por una función de agregación.

GROUP BY <lista_atributos> indica la forma en que deben agruparse las tuplas especificadas en FROM, todos los atributos que aparecen aquí, deben aparecer en la cláusula SELECT.



Funciones de agregación y agrupaciones

Normalmente, las funciones de agregación suelen utilizarse con agrupaciones, de manera que se puede obtener el total, la suma, el mínimo o máximo “para” o “dado” un grupo.

La estructura básica para utilizar una agrupación y función de agregación es:

```
SELECT <lista_atributos>,funcion_agregacion(atributo)
```

```
FROM <nombre_proyecto.nombre_conjunto.nombre_tabla>
```

```
GROUP BY <lista_atributos>;
```



Funciones de agregación y agrupaciones

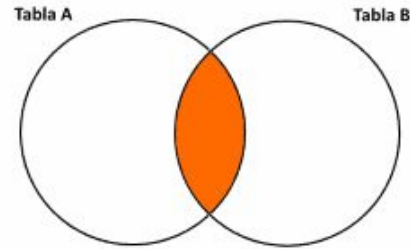
donde:

<lista_atributos> indica los atributos que permitirán generar los grupos de tuplas, por cada combinación de valores diferentes entre estos atributos, se genera un grupo. Todos los atributos que aparecen aquí, deben aparecer en GROUP BY o bien, deben ser utilizados por una función de agregación.

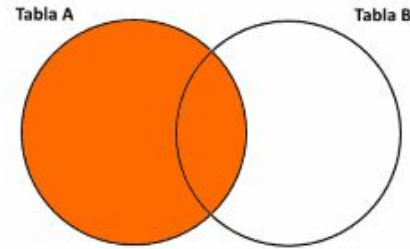
GROUP BY <lista_atributos> indica la forma en que deben agruparse las tuplas especificadas en FROM, todos los atributos que aparecen aquí, deben aparecer en la cláusula SELECT.



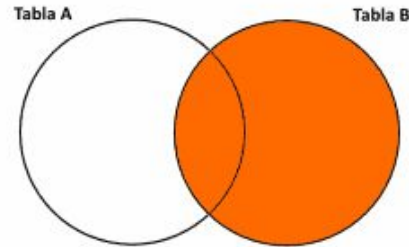
Operaciones JOIN



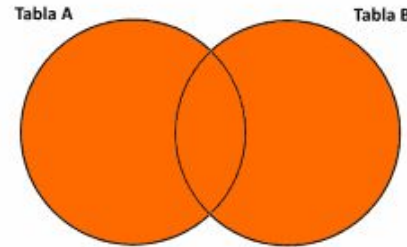
INNER JOIN



LEFT OUTER JOIN



RIGHT OUTER JOIN



FULL OUTER JOIN



Operaciones JOIN

CROSS JOIN

El resultado de esta operación será un producto cruz entre las tuplas de ambas tablas, esto es, todas las tuplas de la tabla A se relacionarán con todas las tuplas de la tabla B.



Operaciones JOIN

LEFT OUTER JOIN / LEFT JOIN

Es la operación de tipo JOIN en la que se mantendrán en el resultado todas las tuplas de la tabla especificada a la izquierda del operador LEFT JOIN. Esta operación requiere especificar una condición en la cláusula ON, esta condición será a través de la cual se relacionarán las tuplas de ambas tablas, esta condición operará comparando los valores de la condición y siempre que se cumpla la condición, se relacionarán las tuplas. Cuando una tupla de la tabla izquierda no tenga correspondencia con una tula de la tabla derecha, se asignarán valores nulos en la consulta de salida a los atributos correspondientes a la tabla B.



Operaciones JOIN

RIGHT OUTER JOIN / RIGHT JOIN

Es la operación de tipo JOIN en la que se mantendrán en el resultado todas las tuplas de la tabla especificada a la derecha del operador RIGHT JOIN. Esta operación, al igual que la operación LEFT JOIN, requiere especificar una condición en la cláusula ON, esta condición será a través de la cual se relacionarán las tuplas de ambas tablas, esta condición operará comparando los valores de la condición y siempre que se cumpla la condición, se relacionarán las tuplas.



Operaciones JOIN

FULL OUTER JOIN / FULL JOIN

Es la operación de tipo JOIN en la que se mantendrán en el resultado todas las tuplas de ambas tablas especificadas en la operación. Esta operación es una combinación entre LEFT JOIN y RIGHT JOIN; requiere especificar una condición en la cláusula ON, que establecerá la relación entre las tuplas de ambas tablas y operará comparando los valores de la condición. Siempre y cuando se cumpla la condición, se relacionarán las tuplas.



Operaciones JOIN

INNER JOIN / JOIN

Es la operación de tipo JOIN en la que se mantendrán en el resultado únicamente las tuplas que en la condición especificada en ON sean verdaderas. La condición en ON operará comparando los valores que se definieron y siempre que se cumplan, se relacionarán las tuplas y se mantendrán en el resultado.



Operaciones JOIN

La estructura básica para utilizar una operación de tipo JOIN es:

SELECT <lista_atributos>

FROM <nombre_proyecto.nombre_conjunto.nombre_tablaA> OPERACIÓN
<nombre_proyecto.nombre_conjunto.nombre_tablaB> ON
<nombre_tablaA.atributo> = <nombre_tablaB.atributo>;

Operaciones JOIN



donde:

nombre_tablaA y nombre_tablaB son las tablas que se espera relacionar

OPERACIÓN es un tipo de operación JOIN entre CROSS JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, JOIN

ON permite indicar la condición de relación de tuplas, en CROSS JOIN no se utiliza, para el resto de operaciones JOIN sí

atributo especifica los atributos de la tablaA y tabla B que se utilizarán como comparación de relación, si los valores son iguales, las tuplas se relacionan



Funciones de ventana

Permiten aplicar una función de agregación a un bloque de tuplas o al conjunto completo de la relación especificada en FROM, no requiere uso de GROUP BY.

Existen funciones específicas que pueden utilizarse como “funciones de agregación” y no requieren parámetro alguno, estas funciones son **ROW_NUMBER()** y **RANK()**.

ROW_NUMBER() permite asignar un valor a cada tupla del conjunto especificado en OVER y en caso de definir PARTITION BY, se asignará el número con base en cada partición

RANK() permite asignar una posición dado el valor de un atributo, requiere el uso de ORDER BY



Funciones de ventana

La estructura básica para utilizar una función de ventana es:

SELECT <lista_atributos>, **funcion_agregación(atributo)OVER(),**

funcion_agregación OVER(PARTITION BY atributo),

**funcion_agregación(atributo) OVER(PARTITION BY atributo ORDER BY
atributo)**

FROM <nombre_proyecto.nombre_conjunto.nombre_tabla>;



Funciones de ventana

donde

`funcion_agregación(atributo)OVER()` permite calcular el resultado de la función de agregación “sobre”/considerando todas las tuplas de FROM

`funcion_agregación OVER(PARTITION BY atributo)` permite calcular el resultado de la función de agregación para cada partición o “subgrupo” formado por los valores únicos del atributo especificado en PARTITION BY

* función de agregación puede ser sustituido por `ROW_NUMBER()` o `RANK()`



Funciones de ventana

donde

`funcion_agregación(atributo) OVER(PARTITION BY atributo ORDER BY atributo)` permite calcular el resultado de la función de agregación para / “sobre” cada partición o “subgrupo” formado por los valores distintos del atributo especificado en `PARTITION BY` y además, ordena cada partición con base en el atributo y sentido especificado en `ORDER BY` de `PARTITION`

* función de agregación puede ser sustituido por `ROW_NUMBER()` o `RANK()`



Expresiones comunes de tabla

Permiten definir, a través de una consulta, una tabla temporal virtual que puede ser utilizada por una consulta, sin embargo, el CTE debe definirse y utilizarse durante el mismo bloque de ejecución.

La estructura básica para utilizar un CTE es:

```
WITH cte_nombre AS (  
    CONSULTA  
)  
SELECT *  
FROM cte_nombre;
```

Expresiones comunes de tabla



donde:

WITH AS () forman la sintaxis en SQL para definición de un CTE, entre paréntesis debe asignarse la consulta que formará el CTE

CONSULTA es la consulta cuyo resultado formará o será el contenido del CTE

cte_nombre es el nombre con el que se podrá referenciar al resultado de la CONSULTA definida en el cuerpo del CTE

SELECT *

FROM cte_nombre; es la consulta que utiliza de manera inmediata el CTE



Vistas

Permiten definir una tabla a partir de una consulta, la vista se puede utilizar como cualquier otra tabla del conjunto de datos y no almacena datos.

La estructura básica para crear una vista es:

```
CREATE VIEW conjunto.vw_nombre AS (  
    CONSULTA  
);
```



Vistas

donde:

`CREATE VIEW AS ( );` forman la sintaxis en SQL para definición de una vista, entre paréntesis debe asignarse la consulta que la definirá

`conjunto` indica el conjunto de datos en el que se creará la vista

`vw_nombre` es el nombre que se le dará a la vista

`CONSULTA` es la consulta que definirá “el contenido” de la vista, esta consulta se ejecutará siempre que la vista se llame



Vistas Materializadas

Permiten definir una vista con datos. En BigQuery, deben incluir una función de agregación y hacer referencia a una única tabla en la cláusula FROM.

La estructura básica para crear una vista materializada es:

```
CREATE MATERIALIZED VIEW conjunto.vw_nombre AS (  
    CONSULTA  
);
```



Vistas Materializadas

donde:

`CREATE MATERIALIZED VIEW AS ( );` forman la sintaxis en SQL para definición de una vista materializada, entre paréntesis debe asignarse la consulta que la definirá
conjunto indica el conjunto de datos en el que se creará la vista

`vwm_nombre` es el nombre que se le dará a la vista materializada

`CONSULTA` es la consulta que definirá “el contenido” de la vista, esta consulta se ejecutará siempre que la vista se llame, **NO** debe incluir `UNION ALL`, `UNION DISTINCT` ni `JOIN`



Funciones definidas por el usuario UDF

Permiten definir una rutina para que permita devolver un valor, pueden recibir parámetros y los parámetros pueden ser atributos de una relación especificada en FROM.

La estructura básica para crear una función definida por el usuario:

```
CREATE FUNCTION conjunto.udf_nombre()  
AS(  
    RUTINA  
);
```




Funciones definidas por el usuario UDF

donde:

CREATE FUNCTION AS(); forman la sintaxis en SQL para definición de una función definida por el usuario

conjunto indica el conjunto de datos en el que se creará la función

udf_nombre es el nombre que tendrá la función

() permite indicar parámetros, en caso de ser vacío, significa que la función no recibe parámetro alguno, si se desea definir parámetros, debe indicarse el nombre del parámetro y su tipo de dato

RUTINA define la rutina que se llevará a cabo por la función, puede utilizar en caso de ser necesario, los parámetros que alimentan a la función, NO puede ser una consulta



Procedimientos

Permite definir una rutina que entregará un resultado, puede incluir una consulta misma que podrá utilizar parámetros y la convertirá en consulta dinámica.

La estructura básica para crear un procedimiento es:

```
CREATE OR REPLACE PROCEDURE conjunto.sp_nombre()
```

```
BEGIN
```

```
    RUTINA
```

```
END;
```

Procedimientos



donde:

CREATE OR REPLACE PROCEDURE forman la sintaxis en SQL para definición de un procedimiento

conjunto indica el conjunto de datos en el que se creará el procedimiento

sp_nombre es el nombre que tendrá el procedimiento

() permite indicar parámetros, en caso de ser vacío, significa que el procedimiento no recibe parámetro alguno, si se desea definir parámetros, debe indicarse el nombre del parámetro y su tipo de dato

BEGIN END; delimitan el cuerpo del procedimiento, forman parte de la sintaxis para definición de un procedimiento

RUTINA define la rutina que se llevará a cabo por el procedimiento, puede utilizar en caso de ser necesario, los parámetros que alimentan al mismo